CSE311 - ARTIFICIAL INTELLIGENCE PROJECT REPORT

SAKSHI

Multi Camera Vehicle Analytics and Evidence Review

An AI based system for vehicle detection, plate review and auditable observations

Submitted by

Student name	Roll number
A Sri Dathan	2024BCS009
Akshay VS	2024BCD0015
Shivnash Neekhra	2024BCS0023
Bhardwaj B Chand	2024BCS0059
Srimoneyshankar Ajith	2024BCS0055

Department of Computer Science and Engineering
Indian Institute of Information Technology Kottayam

Course Artificial Intelligence Mini Project
Academic Year 2026–2027

Report Notes and Acknowledgement

This report uses SAKSHI as the student-facing project title. The repository, source modules and validation records identify the software as AI26. SAKSHI is used for the coursework submission; AI26 is retained when referring to repository paths and recorded measurements.

We acknowledge the authors of the research papers and maintainers of the libraries and model packages listed in the References. The project uses pretrained model assets and demonstration footage; it does not claim that the team trained the detector or OCR model from scratch.

AI tools assisted with repository analysis and preparation of this report. The team remains responsible for checking the code, sources, dataset permissions, measurements and all statements before submission. Each member should be able to explain the methods and limitations described here.

Contents

- Abstract
- 1 Introduction
- 2 Problem Statement and Objectives
- 3 Literature Review
- 4 Proposed System and Methodology
- 5 System Implementation
- 6 Data and Evaluation Method
- 7 Results and Discussion
- 8 Limitations and Ethical Considerations
- 9 Future Scope and Conclusion
- References
- Appendix A Reproducibility and Viva Preparation

Project status at a glance

Table 0.1 *What the available evidence supports*

Area	Status in this report
AI pipeline	Pretrained vehicle/person detection and a separate plate detector plus OCR path are implemented.
System behavior	A local nine-source replay profile and a CPU detector benchmark are recorded.
Accuracy	No independent held-out labels are available; detection and plate accuracy are not established.
Cross-camera identity	Journey candidates remain unresolved without synchronized capture clocks and validated locations.

Abstract

SAKSHI (repository name AI26) is a multi-camera vehicle analytics system that converts recorded traffic footage into reviewable observations. Detection of vehicles and people uses pretrained YOLOv8n, followed by a two-stage licence-plate recognition path: a plate detector and optical character recognition. A bounded scheduler samples all available cameras, and a camera-local motion-and-box tracker associates detections over time. Plate readings pass through normalization, confidence and format checks, and a three-read per-track consensus, so uncertain or conflicting readings are excluded from identity queries. A FastAPI backend stores observations and evidence metadata in SQLite, and a React and TypeScript dashboard supports monitoring, deterministic queries, local rules and evidence export.

In the latest nine-camera replay, 349 sampled frames were processed in 298.011 seconds (1.171 FPS) with zero failed inference attempts. This fell short of the configured 2.0 FPS budget. A separate detector-only benchmark on 60 unlabeled calibration frames measured median model-forward latency of 92.834 ms for PyTorch FP32 and 55.782 ms for static INT8 ONNX.

These are execution measurements only. The repository has no independent held-out labeled dataset, so the report makes no claims about detector precision, OCR accuracy or cross-camera identity matching. It documents the implemented system, small exploratory reviews, an evaluation plan and remaining evidence gaps.

Keywords: vehicle detection, licence-plate recognition, OCR, tracking, multi-camera video, evidence integrity

1 Introduction

1.1 Background

Fixed cameras collect long video streams, but an investigator may need only a few observations: a vehicle class, a possible plate reading, a camera and an approximate time. Searching every recording manually takes time and makes repeated review difficult. Computer vision can reduce footage to candidate observations that a person can inspect, while a data service can preserve how each observation was produced.

1.2 Motivation

A detector can find a car in a frame, but operational questions often involve repeated observations across time. Plate OCR can misread a small or blurred registration, and a replay timestamp can be mistaken for original capture time. SAKSHI addresses this wider workflow by retaining raw reads, recording model provenance, marking uncertain identities and separating measured system behavior from unmeasured recognition accuracy.

1.3 Why artificial intelligence is relevant

The project uses learned visual models where hand-written rules are weak: object detection identifies vehicles in varied scenes, and OCR converts a plate crop into a character sequence. Tracking and consensus organize repeated predictions. These later stages are mostly deterministic algorithms rather than newly trained models. This distinction matters: the AI contribution is pretrained detection and OCR integrated with rule-based association, validation and review.

1.4 Project scope

The current implementation processes recorded sources and can be configured for camera streams. Its demonstrated evidence is local replay and software behavior. The system does not establish city-wide vehicle identity, calibrated speed, congestion or production surveillance readiness. These boundaries are treated as design and evaluation constraints throughout the report.

1.5 Report organization

Section 2 defines the problem and objectives. Section 3 reviews detection, tracking, OCR and quantization work. Sections 4 and 5 describe the proposed data flow and implementation. Sections 6 and 7 document inputs and results. Sections 8 and 9 discuss limitations, next steps and conclusions.

2 Problem Statement and Objectives

2.1 Problem statement

Given video from several fixed cameras, how can a system detect vehicles, propose plate readings, preserve the history and provenance of each observation, and help an investigator review candidate activity without turning a low-quality prediction into an unsupported identity?

2.2 Objectives

- Run one shared background processor that samples available cameras under a configurable global frame budget.
- Use pretrained computer-vision models to detect vehicles and, when enabled, people for local privacy masking.
- Detect and read plate regions, normalize candidate text, and retain raw, invalid, low-confidence and ambiguous observations for review.
- Associate detections within a camera using motion and bounding-box cues, while retiring tracks when assignment is ambiguous.
- Persist observations, camera metadata, model hashes, image derivatives, review actions and audit records.
- Provide a dashboard for camera monitoring, bounded queries, local rules, analytics and evidence exports.
- Create reproducible data-manifest and evaluation tools, and report what they can and cannot measure.

2.3 Scope and measurable objectives

Table 2.1 *Objective and evidence boundary*

Objective	Evidence available	Open criterion
Shared multi-camera processing	A recorded nine-source 300-second replay profile.	Sustained target-device rate and live stream behavior.
Vehicle and plate AI	Pretrained models load and run; a small review audit exists.	Held-out detector and exact-plate metrics.
Tracking	Deterministic test scenarios exercise association and ambiguity rules.	Field ID-switch and fragmentation rates.
Evidence handling	Hash references, immutable snapshots and optional signing are implemented.	Independent provenance and operational key controls.
Cross-camera journey	Clock and route constraints are represented in code.	Verified clocks, surveyed locations and labeled route pairs.

2.4 Success criteria

For this coursework report, success means that the system demonstrates a meaningful AI application, keeps uncertainty visible, produces reproducible software measurements, and makes no claims stronger than its labels support. An accuracy claim requires a complete labeled test split that is independent by vehicle, camera, location and session.

3 Literature Review

SAKSHI combines established methods rather than proposing a new detector or recognition architecture. The literature informs the choice of compact visual models, association algorithms and evaluation discipline. Table 3.1 relates this work to the implementation.

3.1 Object detection

Redmon et al. framed detection as a single network that predicts bounding boxes and classes from an image, establishing the You Only Look Once family of real-time detectors [1]. The project uses the Ultralytics YOLOv8 nano checkpoint as a pretrained detector for vehicle classes and, in privacy mode, people [2]. The project's CPU benchmark measures execution only; the repository provides no labeled result showing how well the checkpoint detects vehicles in these clips.

3.2 Tracking by detection

SORT uses motion estimates and the Hungarian assignment algorithm to link detections efficiently [3]. DeepSORT adds a learned appearance metric to help maintain identities through occlusion [4]. SAKSHI instead uses a camera-local motion-and-box cost with global assignment and an explicit ambiguity margin. It has no learned appearance embedding, so its track IDs are local associations rather than persistent vehicle identities.

3.3 Licence-plate detection and text recognition

Laroca et al. describe an automatic plate-recognition pipeline based on YOLO detection and downstream recognition [5]. Fast-ALPR exposes a plate detector followed by OCR [6]. Scene-text research such as CRNN treats recognition as transcription of an image sequence without first segmenting every character [7]. The selected upstream global-plate OCR asset is documented as a MobileViT-v2 model [8, 9]. Its upstream training data and vendor-reported results are not measurements of this project or proof of Indian-plate accuracy.

3.4 Efficient inference and evaluation

Integer quantization can reduce model size and inference time with lower-precision arithmetic, but the result depends on calibration and model behavior [10]. SAKSHI benchmarks a static INT8 export against FP32 on the same unlabeled sample frames. This comparison reports speed and output agreement, not accuracy. The evaluation tools also report rate intervals using the Wilson approach [11], but confidence intervals cannot repair missing or unrepresentative labels.

3.5 Research gap addressed

The project contribution is an integrated review workflow around uncertain predictions: separate raw and supported plate readings, model hashes, persistent observation histories, audit records, a query interface and evidence exports. These engineering safeguards do not make the perception models accurate. They make it easier to inspect how a result was produced and to decline an identity claim when evidence is weak.

Table 3.1 *Relevant methods and their role in SAKSHI*

Work or method	Main idea	Relationship to this project
YOLO [1, 2]	Single-stage detection for image regions and classes.	Pretrained YOLOv8n provides vehicle and person detections; local accuracy is unmeasured.
SORT [3]	Motion-based online association with global assignment.	The project uses assignment with custom motion, geometry and ambiguity rules.

Work or method	Main idea	Relationship to this project
DeepSORT [4]	Adds learned appearance association to SORT.	Comparison only; SAKSHI has no appearance embedding or cross-camera ReID.
YOLO-based ALPR [5]	Detect plates, then recognize characters.	Supports the modular plate detector plus OCR structure.
CRNN [7]	End-to-end sequence recognition for scene text.	OCR background; not the architecture selected by the repository.
MobileViT v2 [8, 9]	Lightweight vision-transformer family and upstream plate OCR asset.	Global pretrained OCR component; Indian-scene validation remains open.
Integer quantization [10]	Lower-precision execution can trade size and latency against output behavior.	Motivates the INT8 comparison; no accuracy claim follows.

3.6 Implications for the design

Across these methods, SAKSHI's main contribution is the way it joins separate perception and review stages. A detector proposes boxes, a tracker associates them within one camera, and OCR proposes plate text; each stage has different failure modes and needs its own evidence. The implementation therefore keeps plate and vehicle tracks separate, preserves rejected readings for review, and avoids treating track assignment as a learned identity. The literature also highlights a gap the current project has not closed: representative, independently labeled evaluation. Before deployment or a model comparison, the team would need held-out data that reflects its cameras and plate conditions, with uncertainty and abstention reported alongside recognition metrics.

4 Proposed System and Methodology

4.1 System overview

The system has six stages: camera-source resolution, shared sampling, AI inference, camera-local association and plate review gates, persistent storage, and operator-facing services. A single processor owns inference. Browser requests read cached frames and API results, so adding viewers does not create more model workers. Figure 4.1 shows the data flow.

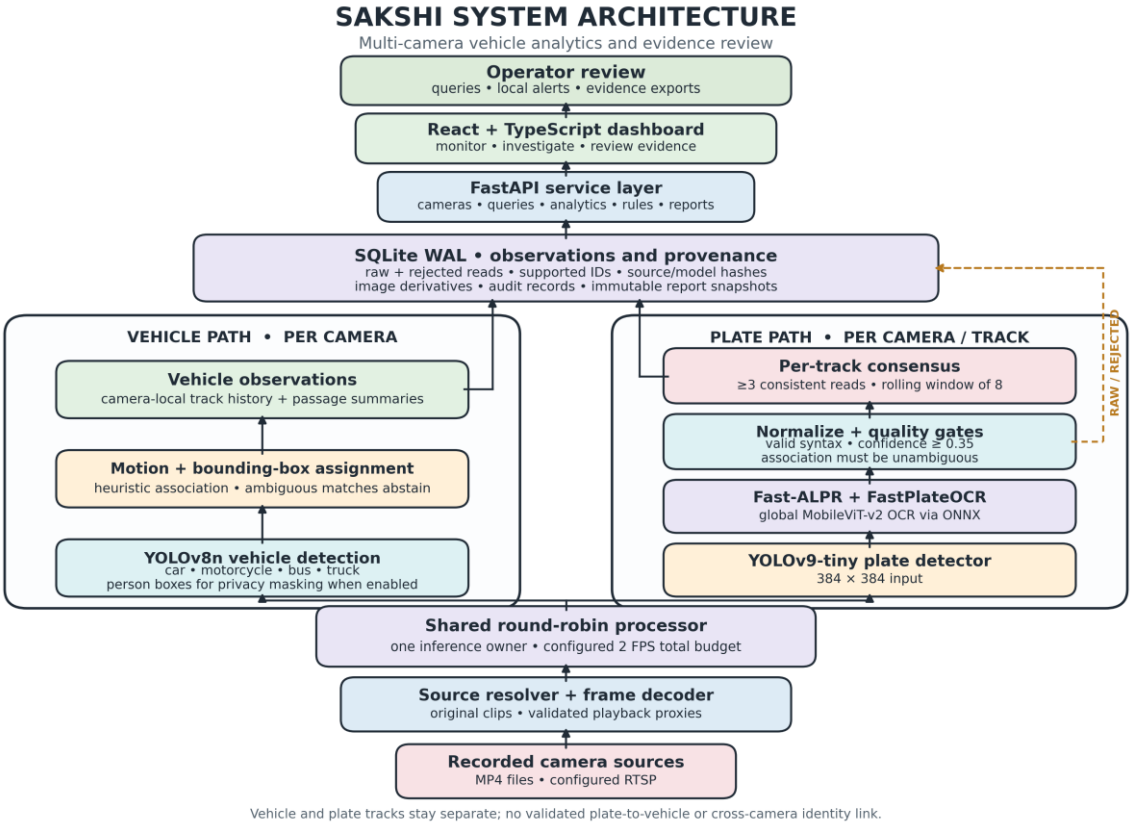


Figure 4.1 *SAKSHI system architecture and observation flow*
Source: authors' diagram based on the repository implementation and API documentation.

4.2 Inputs and frame preparation

Inputs can be recorded MP4 sources or configured RTSP sources. Source management distinguishes original footage from browser-compatible playback copies, records file digests and validates selected media before activation. The processor decodes sampled frames and maintains replay position separately from processing time. The vehicle model receives a 640 × 640 model input; the plate detector uses its configured 384-pixel input.

The default scheduler has a 2-frame-per-second total budget, not 2 frames per second per camera. It cycles across available sources. Missing or retrying sources do not consume inference slots. Recorded video playback is separate from the low-rate sampled detection view.

4.3 Vehicle and person detection

The backend loads the YOLOv8n checkpoint lazily and runs it on CPU in the documented evaluation configuration. The detector filters for COCO car, motorcycle, bus and truck classes. Person boxes are requested when privacy masking is enabled; local pixelation is applied before sampled output is shared. Confidence thresholds control candidate generation, but model confidence is not a calibrated probability and cannot be treated as accuracy.

4.4 Camera-local tracking

Each camera keeps its own active tracks. For a candidate track and detection, the cost combines predicted center displacement, bounding-box overlap and change in box area. Class mismatch, implausible size change and distance outside a gate reject a pair. The current tracker uses:

$$\text{cost} = 0.75 \times \text{normalized distance} + 0.20 \times (1 - \text{IoU}) + 0.05 \times \text{normalized area change}$$

The implementation solves a minimum-cost assignment with an unmatched option. Near-equal competing matches are marked ambiguous and implicated old tracks are retired. These costs are heuristic association scores, not identity probabilities. Track IDs reset with camera sessions; no appearance model links vehicles across cameras.

4.5 Plate recognition and abstention

The plate branch runs a YOLOv9-tiny 384 × 384 plate detector and the FastPlateOCR global MobileViT-v2 OCR asset through ONNX Runtime. The source code normalizes candidate strings to supported Indian registration syntax and records original OCR text, canonical value, confidence and normalization actions. Plate tracks are separate from vehicle tracks; the code does not establish a validated association between them.

A candidate below the 0.35 OCR confidence threshold, outside the accepted format, or associated with an ambiguous track remains in observation history and is excluded from supported plate queries. For eligible reads, consensus is scoped to camera and track, keeps up to eight recent readings, and supports a plate only after at least three consistent readings. Conflicting strings remain conflicting; character-level suggestions are review-only.

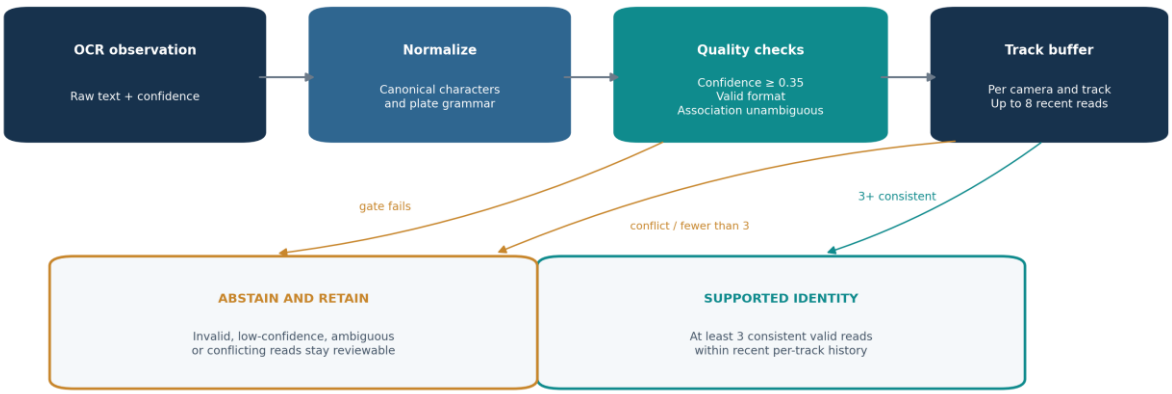


Figure 4.2 *Plate-reading decision path and abstention behavior*
Source: authors' diagram based on backend/inference.py and backend/fusion.py.

4.6 Time and cross-camera journeys

An observation can carry replay position, processing completion time and, when supplied by an operator, an original-time claim marked unverified. Replay position is not the time when a real-world camera recorded the scene. The current runtime does not emit a synchronized_capture_clock time basis, so physical travel-time calculations across cameras remain unresolved. Candidate links require route and time constraints and still represent associations for review, not proof of identity.

4.7 Data and evidence flow

The processor stores frame-level reads and camera-local passage summaries in SQLite with a write-ahead log. Observations record source, session, frame position, model and pipeline-component digests. The

system attempts SHA-256-referenced JPEG derivatives for sampled reads and records missing image coverage when capture or storage fails. Evidence reports snapshot the selected interval and supporting reads. A hash can detect a change to stored report bytes; an optional Ed25519 signature can verify a signed report against an independently trusted public key. Neither proves that a scene or OCR result is true.

5 System Implementation

5.1 Software components

Inference ownership is separate from interface activity. The backend coordinates sources and models, validates API requests, stores records and applies review rules. The web client requests data and cached images. Evaluation commands operate on explicit records and manifests instead of silently treating demonstration footage as ground truth.

Table 5.1 *Main code modules and responsibilities*

Component	Repository area	Purpose
Inference	backend/inference.py	Loads YOLO and Fast-ALPR, runs visual models and records plate confidence.
Scheduling and sources	backend/processing.py; backend/cameras.py	Owns round-robin sampling, decode state, health counters and source availability.
Tracking and fusion	backend/tracking.py; backend/fusion.py	Associates detections within a camera and applies per-track OCR consensus.
Persistence and API	backend/events.py; backend/main.py	Stores observations and rules; exposes camera, query, analytics and report endpoints.
Web dashboard	frontend/	React and TypeScript camera wall, investigations, analytics, alerts and evidence views.
Evaluation tools	evaluation/	Manifest checks, labeled-record metrics, ablations, runtime profiling and frame-review preparation.

5.2 Libraries and model assets

The current model card records Python 3.12, PyTorch 2.14 CPU, Ultralytics 8.4.155, ONNX Runtime 1.30.0, Fast-ALPR 0.4.0, FastPlateOCR 1.1.0 and Open Image Models 0.6.0. The broader application uses OpenCV and NumPy for video and image operations, FastAPI for HTTP services, SQLite for local persistence, and React, TypeScript and Vite for the browser interface.

The vehicle checkpoint and plate detector/OCR assets are identified by cryptographic digests in the model bundle. Provenance identifies the exact files used; it does not establish training data, model rights, validation quality or deployment approval. Model-weight terms should be checked independently before redistribution or deployment.

5.3 Query and review behavior

Queries use a bounded grammar for plate, camera, class and time. Unsupported questions are rejected explicitly. The system has no LLM, generated SQL or external registration lookup. Local watchlists and zone rules create review alerts; a zone alert does not prove unauthorized access.

5.4 Privacy and audit controls

Privacy mode pixelates sampled people and plates and disables raw playback. Optional authentication adds roles, camera and case scopes, CSRF protection, TOTP and access audits. Because authentication is optional and cases share camera-level boundaries, this remains a local coursework system, not a security certification.

Hash-chained audits and optional Ed25519 signatures can help detect changes to checkpoints and evidence reports. Independent storage, key management and source authenticity remain necessary.

6 Data and Evaluation Method

6.1 Available video and review inputs

The repository has six bundled playback clips, CAM-01 through CAM-06, at 960 × 540 and a reported 25 frames per second. Three additional managed original sources, CAM-07 through CAM-09, are represented in local validation records at 1920 × 1080. The first six sources use playback fallback files in the prepared review set, while the latter three use managed originals. Camera file names do not establish where or when footage was captured.

Table 6.1 *Inputs represented in the available review records*

Sources	Role in this report	Known limits
CAM-01 to CAM-06	Bundled playback clips used for local replay and detector calibration.	Not a labeled test set; location, session independence and capture time are unverified.
CAM-07 to CAM-09	Locally managed originals used for selected plate reviews and active replay.	Origin, authorization, timezone and camera metadata are not independently verified.
45-frame review set	Five prediction-blind samples per camera from equal-duration strata.	Prepared 18 October 2026; no human labels were prefilled; not held out.
21 selected plate crops	Small human readability review summarized in validation.	Selected private examples; 16 were unreadable; not representative.

6.2 Dataset preparation and leakage controls

The review-set generator hashes source clips, selects frame positions without inspecting predictions, and records image hashes. The 45-frame set contains five frames from each of nine sources. At preparation time it contained no human vehicle counts or plate text. The evaluation manifest requires clip hashes, explicit labels, train/validation/test membership, and separation by vehicle, camera, location and session. Repeated vehicles or adjacent frames must not be treated as independent test examples.

The repository does not include a complete reviewed train/validation/test dataset for this report. The 45-frame set is a labeling workflow, not proof that a model meets an accuracy target. Six clips may contain correlated views or repeated vehicles; independence is not established.

6.3 Evaluation protocol

When labels are ready, the detector evaluator matches same-class predicted and reference boxes one-to-one at an intersection-over-union threshold of 0.50 and reports precision and recall with denominators. OCR evaluation compares canonical strings exactly and reports correct, incorrect, missing and abstained outputs separately. Tracking evaluation requires camera, session and chronological frame labels, then counts ID switches and visible-track fragmentation. Journey evaluation needs independently validated location and synchronized clock information. Paired ablations require identical held-out samples and exact model provenance across variants.

6.4 Operational metrics

Runtime profiling reports sampled frames per second, processing failures, phase timings, endpoint latency, CPU and resident memory. Model confidence is shown separately from measured accuracy. These measures describe software behavior under recorded conditions; they do not measure the fraction of vehicles or plates recognized correctly.

Table 6.2 *Evidence required before making model-quality claims*

Claim	Required ground truth	Metric reported by project tools
Vehicle detection	Complete class-labeled boxes on held-out frames.	One-to-one class-aware precision and recall at IoU 0.50.
Plate OCR	Reviewed readable plate text with matched crop regions.	Exact string match, coverage, errors and abstentions.
Tracking	Vehicle identities across ordered frames in independent sessions.	ID switches, fragmentation and matched identity counts.
Cross-camera association	Verified locations, synchronized clocks and labeled vehicle pairs.	Link precision/recall and false-candidate rates with uncertainty.
Runtime	Host, model, branch settings and sample window.	Latency, throughput, failures, CPU and memory.

6.5 Interpreting future evaluation results

A fair comparison reports model quality and coverage together. For OCR, exact-string accuracy on readable crops should be paired with a coverage or abstention rate; unreadable and uncertain labels should be shown separately. Otherwise a conservative gate can appear accurate simply by abstaining on hard examples. Detector precision and recall require reference boxes, while tracking metrics require stable identity labels across ordered frames. Every result should include sample counts, exclusions, model hashes and environment details.

These measurements answer different questions: recognition quality on labeled examples, review burden from abstention, and runtime cost under a chosen hardware and scheduler. The current artifacts cover runtime, model-to-model agreement and selected crop observations. Keeping those categories separate makes future results reproducible without overstating today’s evidence.

7 Results and Discussion

Available results measure local runtime, model-forward latency and a small crop-review sample. They do not establish general recognition accuracy. Results below retain the scope recorded in the repository.

7.1 Nine-camera replay profile

A five-minute local replay profile enabled vehicle detection and plate recognition for all nine available sources. The valid counter window lasted 298.011 seconds. The processor completed 349 sampled frames, or 1.1711 frames per second against the configured 2.0 total budget. No inference attempts or health reads failed. Completion counts were balanced at 38 or 39 per camera, but the measured system did not reach its configured rate.

Table 7.1 *Five-minute active-inference profile*

Measure	Observed result	Interpretation
Available sources	9 of 9 processing at start and end	Replay evidence for all configured sources.
Valid counter window	298.011 s	Profile checks found no invalid sample gap.
Completed samples	349; 1.1711 FPS total	Below the configured 2.0 FPS total budget.
Failed attempts	0; health-read failures 0	No failures in this bounded replay window.
CPU usage	163.5% median; 226.9% p95	Process CPU can exceed 100% across multiple cores.
Resident memory	830.2 MiB median; 851.5 MiB p95	Local replay measurement, not a device requirement.
Plate-source overrun counts	CAM-07 36/38; CAM-08 37/38; CAM-09 39/39	Attempts exceeded 500 ms per-attempt target; not dropped source-frame counts.

Source: docs/validation/active-inference-profile-9cams-2026-10-18-300s.json; local 12-logical-CPU Linux host.

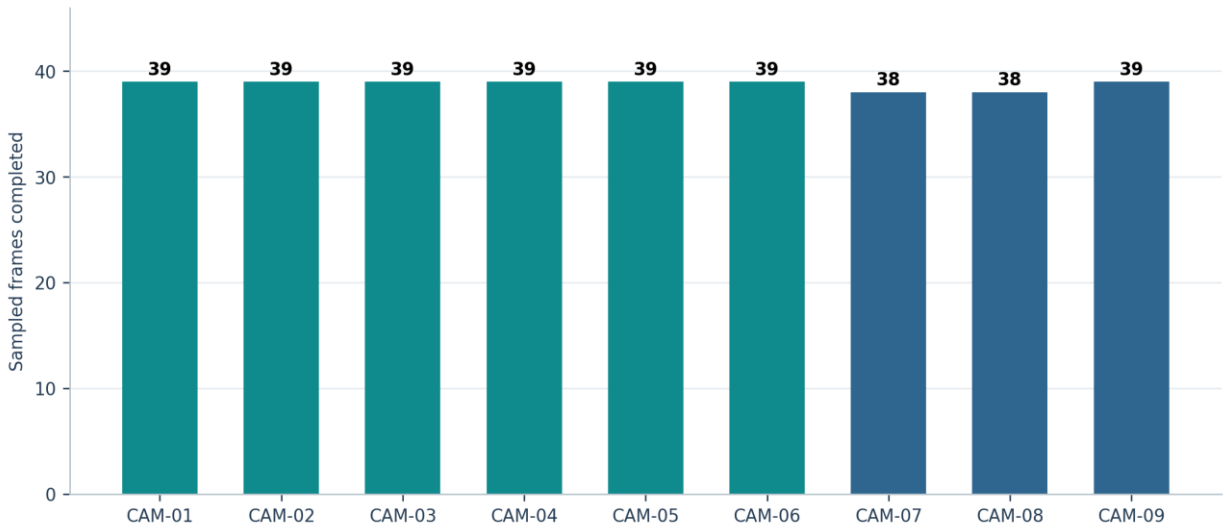


Figure 7.1 *Sampled frames completed by camera in the nine-source replay*
Source: repository validation profile dated 18 October 2026. Counts are sampled outputs, not every decoded source frame.

7.2 Detector runtime comparison

The detector benchmark used sixty sampled frames from the six bundled clips to calibrate a static quantized export. It measured ten model-forward runs after two warm-ups on a 12th-generation Intel Core i5-12450H CPU host. The input was 640 × 640; decoding, preprocessing, non-maximum suppression, rendering, database writes and OCR were outside the latency measurement.

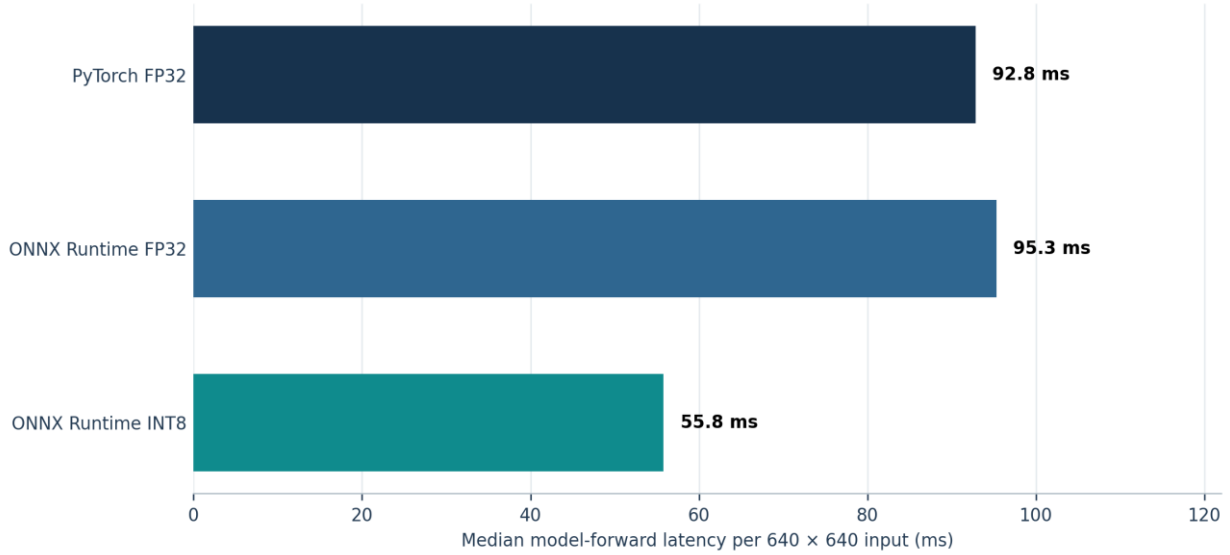


Figure 7.2 *YOLOv8n median model-forward latency by runtime*

Source: docs/validation/edge-benchmark-local-host-2026-10-14.json. Lower latency is faster; this is not end-to-end pipeline time.

Table 7.2 *Detector-only CPU benchmark*

Runtime	Median / p95 latency	Model file size	Sample scope
PyTorch FP32	92.834 / 98.743 ms	6.55 MB	10 measured runs
ONNX Runtime FP32	95.309 / 101.240 ms	12.82 MB	10 measured runs
ONNX Runtime INT8	55.782 / 58.306 ms	3.60 MB	10 measured runs; static QDQ

INT8 reduced median model-forward latency by about 41% relative to FP32 ONNX and reduced the exported file from 12.82 MB to 3.60 MB. In the same sixty unlabeled calibration frames, INT8 matched 15 of 16 FP32 reference detections under class-aware IoU matching at 0.50. The frames had no ground-truth labels, so this is model-to-model agreement, not precision, recall or proof that the INT8 export is accurate. The quantized artifact was not integrated into live inference.

7.3 Selected plate-crop diagnostic review

This is a selected diagnostic sample, not a representative test set. A reviewer assessed 21 crops from CAM-07 through CAM-09: 16 were unreadable, 4 readable and 1 uncertain. Only the 4 readable crops support direct string comparison. In the recorded pipeline results, all four OCR strings differed from reviewer transcriptions and each mismatch was rejected by the configured gates; none became a supported plate ID. This demonstrates abstention on these examples, not OCR accuracy or a false-accept rate.

An offline probe applied 68 crop variants to the four readable crops. One variant matched exactly; five passed format and confidence checks (one exact, four inexact). The probe did not test track association or repeat-read consensus and was not deployed, so it is diagnostic only, not a live-system result or accuracy estimate.

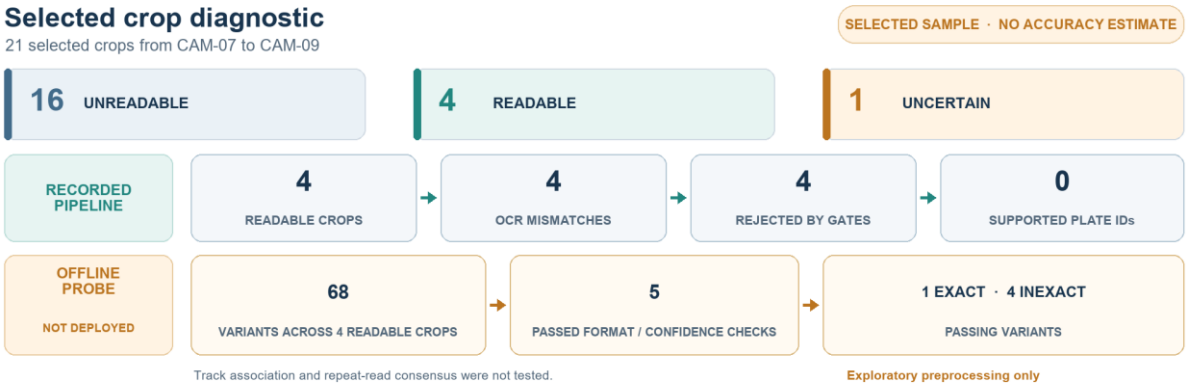


Figure 7.3 *Selected diagnostic sample: reviewer labels, recorded-pipeline outcomes, and a separate offline probe*
Source: docs/validation/ocr-groundtruth-evaluation-2026-10-18.json. Selected sample; not representative and not an accuracy estimate.

Table 7.3 *Summary and limits of the selected crop diagnostics*

Evidence slice	Observed result	What can be concluded
Selected crops	21 total: 16 unreadable, 4 readable, 1 uncertain.	Only 4 support direct string comparison; the sample is not representative.
Recorded pipeline	All 4 readable-crop OCR strings mismatched and were rejected; none became a supported ID.	Shows gate behavior in these cases only; not an accuracy or false-accept rate.
Offline probe	68 variants: 1 exact match; 5 passed format/confidence checks (1 exact, 4 inexact).	Association and repeat-read gates were not tested; exploratory and not deployed.

7.4 What the results support

Table 7.4 *Supported findings and claims still open*

Supported by recorded evidence	Not established by this report
The pipeline completed a bounded nine-source replay without failed attempts.	A sustained 2 FPS rate, live-camera reliability or long-duration recovery.
The runtime shared 38–39 sampled outputs per camera in the window.	Full-rate analytics or equal source-frame coverage.
INT8 ran faster than FP32 ONNX in a detector-only local benchmark.	Ground-truth detector accuracy or safety of model promotion.
Plate gates rejected four mismatched outputs in a small review.	General Indian plate accuracy or a low false-identification rate.
Hashes and optional signatures can verify stored artifact integrity.	Truth of the scene, plate text or independent chain of custody.

7.5 Discussion

The nine-camera replay delivered 1.17 FPS against a 2 FPS budget, and plate-camera inference often exceeded its 500 ms target. End-to-end profiling on intended hardware is needed; one CPU profile cannot establish deployment capacity.

INT8 was faster and smaller than FP32 ONNX, with one disagreement on unlabeled calibration frames. Held-out labels are needed to judge that trade-off. In the selected crop review, all four readable-crop mismatches were rejected by the plate gates, while 16 of 21 crops were unreadable. These are workflow observations, not accuracy evidence.

8 Limitations and Ethical Considerations

The evidence is strongest for implementation and bounded runtime behavior. It is weak for model quality because labels, capture provenance and independent splits are incomplete. These limitations define what the system should not be used to claim.

- No complete held-out dataset is available. Detector precision/recall, plate exact-match accuracy, confidence calibration and field tracking rates are unknown.
- The OCR sample includes only four crops judged readable and comes from selected footage. Rejected mismatches show the abstention gate's behavior on those examples, not a general false-identity rate.
- The 45-frame review set is prepared but unannotated. Some clips may share vehicles or locations, and source metadata does not verify capture time, timezone, camera identity or authorization.
- Track identities are camera-local and use motion and geometry. Plate tracks are not linked to vehicle tracks by validated evidence, and there is no learned cross-camera re-identification.
- Replay timestamps and arrival times are not synchronized capture clocks. Physical speed, route travel time and city-wide unique-vehicle counts cannot be inferred.
- The latest active-inference profile is a five-minute replay on one 12-logical-CPU workstation. It does not establish live RTSP performance, target-edge suitability, multi-hour uptime, thermal behavior or browser playback quality.
- Authentication and local privacy masking are implemented, but the project has not undergone production security, legal or privacy assessment. Footage rights and model-weight redistribution terms must be checked.

8.1 Responsible interpretation

A plate reading can identify a person's vehicle, so even an unresolved OCR candidate is sensitive. The report does not reproduce plate strings or private frame images. A practical deployment would need a documented legal basis, data minimization, retention limits, access monitoring, secure key handling and human review before a consequential decision. A watchlist match or restricted-zone alert is a review prompt, not a finding of wrongdoing.

The system should be demonstrated only with footage the team is authorized to use. If a later report includes sample frames, confirm permission, mask identifying details where appropriate, credit the source and preserve uncertainty labels.

9 Future Scope and Conclusion

9.1 Future work

- Complete an authorized, prediction-blind annotation set with vehicle boxes, readable and unreadable plate labels, camera/session metadata and source rights. Keep vehicles, sessions and locations isolated across train, validation and test splits.
- Run the evaluator on held-out detections and OCR outputs. Report denominators, per-camera results, abstention coverage, error types and confidence intervals. Do not select thresholds on the final test split.
- Use licensed, locally representative plate data to compare the pretrained OCR against a region-specific or fine-tuned model. Promote changes only after paired evaluation on identical held-out samples.
- Measure end-to-end performance on intended hardware with both inference branches enabled. Record cold start, warm latency, throughput, skipped frames, CPU, memory, power, thermal behavior and recovery over longer periods.
- Obtain verified camera locations and synchronized capture clocks before evaluating cross-camera routes. Validate plate-to-vehicle association against human-reviewed sequences or keep the tracks separate.
- Review authentication defaults, privacy policy, data retention and model/data licensing before deployment beyond the coursework demonstration.

9.2 Conclusion

SAKSHI demonstrates an end-to-end AI application that combines pretrained vehicle detection and plate OCR with camera-local tracking, conservative identity support, searchable observations and evidence-oriented storage. The repository also includes a web dashboard, local review rules, provenance records, privacy options and reproducible evaluation tools. The latest nine-camera profile completed 349 samples over a valid 298-second window without failed attempts, but achieved 1.17 FPS against a 2 FPS total budget. Detector-only INT8 profiling was faster than FP32 ONNX, while the OCR review remained too small to estimate accuracy. The result is a working system with explicit uncertainty controls and measurable operational limits. A larger, authorized and independently labeled dataset is required before claiming recognition accuracy or cross-camera identity.

References

Research papers are listed first, followed by upstream software/model documentation and repository evidence. Access dates for changing software documentation are 20 October 2026.

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," CVPR, 2016; arXiv:1506.02640

[2] Ultralytics, "YOLOv8 model documentation and software citation," YOLOv8 documentation

[3] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," WACV, 2016; arXiv:1602.00763

[4] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," 2017; arXiv:1703.07402

[5] R. Laroca et al., "A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector," 2018; arXiv:1802.09567

[6] FastALPR maintainers, "Fast-ALPR API reference," Fast-ALPR reference

[7] B. Shi, X. Bai, and C. Yao, "An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition," 2015; arXiv:1507.05717

[8] S. Mehta and M. Rastegari, "MobileViT: Light-Weight, General-Purpose, and Mobile-Friendly Vision Transformer," ICLR, 2022; arXiv:2110.02178

[9] S. Mehta and M. Rastegari, "Separable Self-Attention for Mobile Vision Transformers," 2022; arXiv:2206.02680

[10] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," CVPR, 2018, pp. 2704–2713; CVPR Open Access

[11] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927; DOI: 10.1080/01621459.1927.10502953

[12] FastPlateOCR maintainers, "Inference model zoo," model names, architecture notes and upstream result links; FastPlateOCR model zoo

[13] Open Image Models maintainers, "Plate detection model catalogue"; Open Image Models

[14] FastAPI, "SQL databases," official documentation; FastAPI SQL documentation

[15] AI26 repository, README and implementation notes, including docs/IMPLEMENTATION.md, docs/EVALUATION_REGISTER.md, evaluation/README.md and docs/PROGRESS.md. Repository snapshot reviewed 20 October 2026.

[16] AI26 validation artifacts: docs/validation/active-inference-profile-9cams-2026-10-18-300s.json; docs/validation/edge-benchmark-local-host-2026-10-14.json; docs/validation/ocr-groundtruth-evaluation-2026-10-18.json; docs/validation/frame-review-preparation-2026-10-18.json.